Web Vitals, Tree Shaking & Bundle Optimization

5 min read Updated 16 Jul 2026 Topic 29 of 62

5.5 – Web Vitals, Tree Shaking & Bundle Optimization

🎯 What Interviewers Actually Ask

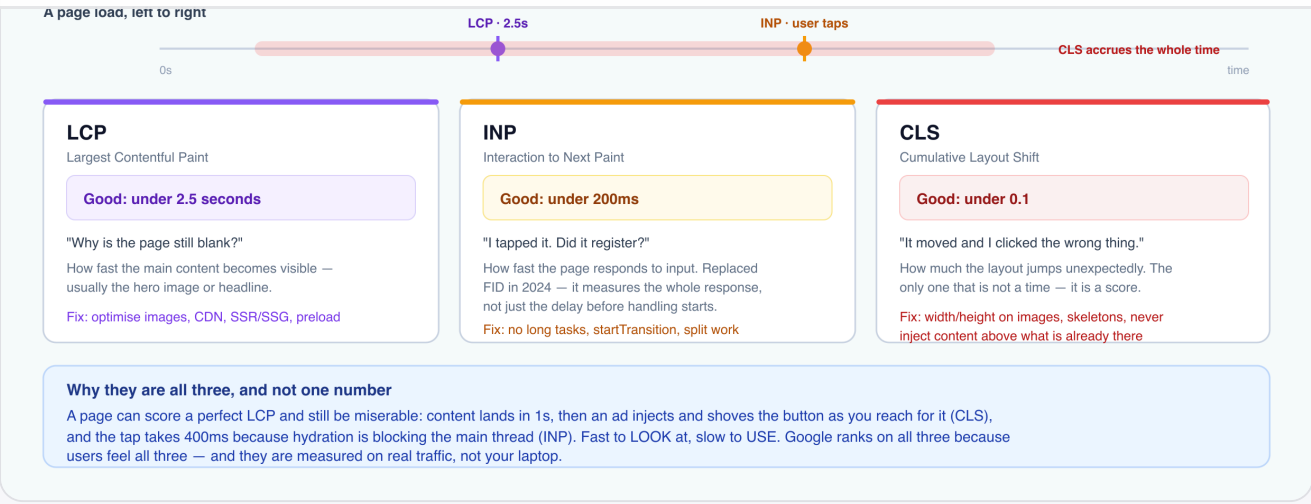
Question	Level	Frequency
"What are Core Web Vitals?"	🟡🔴	★★★☆☆
"What is LCP, FID, CLS?"	🔴	★★★☆☆
"What is tree shaking?"	🟡	★★★☆☆
"What is the difference between CSR and SSR?"	🟡🔴	★★★★☆
"How do you reduce bundle size?"	🟡🔴	★★★★☆

Core Web Vitals

Google uses Core Web Vitals as ranking factors for search. They measure real user experience, not just server response time.

Metric	What it measures	Good threshold	How
LCP (Largest Contentful Paint)	How fast the main content becomes visible	Under 2.5 seconds	Opti
INP (Interaction to Next Paint)	How fast the page responds to user input	Under 200ms	Avoi
CLS (Cumulative Layout Shift)	How much the page layout shifts unexpectedly	Under 0.1	Set e

INP replaced FID (First Input Delay) in March 2024 as a Core Web Vital. Some interviewers still ask about FID, but INP is the current standard.



Client-Side Rendering vs Server-Side Rendering

This is one of the most common senior interview questions.

	CSR (Client-Side Rendering)	SSR (Serv
How it works	Browser downloads JS bundle, executes it, then renders the page	Server ren
Initial load	Slower (blank page until JS loads and executes)	Faster (HT
SEO	Poor (search crawlers may not execute JS)	Good (full
Subsequent navigation	Fast (no server round trips, client-side routing)	Depends
Server cost	Low (static hosting works)	Higher (se

Framework	Create React App, Vite	Next.js, Re

Hydration is the process where React attaches event listeners and state to the server-rendered HTML, making it interactive. The HTML is visible immediately, but the page becomes fully interactive only after hydration completes.

Bundle Size Optimization Strategies

1. Analyze what's in your bundle

Run a bundle analyzer (covered in 5.2) to see what's actually large.

2. Replace heavy dependencies

Heavy library	Lightweight alternative	Size savings
moment.js (72KB)	dayjs (2KB) or date-fns (tree-shakeable)	~70KB
lodash (72KB)	lodash-es (tree-shakeable) or native JS	Up to 72KB
axios (14KB)	Native fetch API	14KB
classnames (2KB)	Template literals or clsx (0.5KB)	Small but adds up

3. Use tree shaking properly

Import only what you use, using named imports from ES module packages.

JSX Copy

```jsx
// ❌ Imports the entire library
import _ from 'lodash';
_.debounce(fn, 300);

// ✅ Imports only debounce (tree-shakeable with lodash-es)
import { debounce } from 'lodash-es';
debounce(fn, 300);

// ✅ Or import the specific module directly
import debounce from 'lodash/debounce';
debounce(fn, 300);
```

4. Compress and minify

Modern bundlers (Vite, Webpack 5) handle minification automatically in production builds. Ensure your server sends assets with gzip or brotli compression.

5. Use a CDN for static assets

Serve JavaScript, CSS, images, and fonts from a CDN (Cloudflare, AWS CloudFront, Vercel Edge) to reduce latency for users around the world.

6. Optimize images

Images are often the largest assets on a page. Use modern formats (WebP, AVIF), responsive sizes (srcset), and lazy loading (`loading="lazy"` attribute).

JSX Copy

```jsx
<img
  src="/hero.webp"
  alt="Hero image"
  width={1200}
  height={600}
  loading="lazy"
```

Measuring Web Vitals in React

JSX Copy

```jsx
import { onLCP, onINP, onCLS } from 'web-vitals';

onLCP(metric => console.log('LCP:', metric.value));
onINP(metric => console.log('INP:', metric.value));
onCLS(metric => console.log('CLS:', metric.value));
```

Or use Lighthouse in Chrome DevTools (Ctrl+Shift+I, then Lighthouse tab) for a full audit with actionable recommendations.

Common Mistakes

Mistake 1: Ignoring images. Developers spend hours optimizing JavaScript bundle size while serving uncompressed 5MB hero images. Images are usually the biggest payload.

Mistake 2: Using CSR for SEO-critical pages. Search engines may not execute JavaScript, so CSR pages can appear empty to crawlers. Use SSR or SSG for pages that need to rank in search results.

Mistake 3: Not setting image dimensions. Without explicit width and height, images cause layout shifts (CLS) when they load. Always set dimensions or use aspect-ratio CSS.

Interview Answer

"I measure real user experience: LCP for loading speed, INP for interactivity, and CLS for visual stability. For loading performance, I optimize the bundle with tree shaking, code splitting, and replacing heavy libraries like moment.js with lighter alternatives. I analyze the bundle to find what's actually large. For SEO-critical pages, I use SSR or SSG with Next.js instead of pure client-side rendering. For images, I use modern formats like WebP, lazy loading, and explicit dimensions to prevent layout shifts. I serve static assets from a CDN with compression. And I always measure with Lighthouse before and after changes to validate the improvements."

✅ Key Takeaways

- **Core Web Vitals:** LCP (loading), INP (interactivity), CLS (visual stability). Google uses them for SEO ranking.
- **CSR** for dashboards/internal tools. **SSR** for SEO-critical pages. **SSG** for static content.
- **Tree shaking** removes unused code. Requires ES module imports (not require).
- **Replace heavy dependencies:** moment.js with dayjs, full lodash with lodash-es or native JS.
- **Images are usually the biggest payload.** Use WebP/AVIF, lazy loading, explicit dimensions.
- **Bundle analysis** first, optimization second.
- **CDN + compression** for all static assets.
- **Measure with Lighthouse and web-vitals library** before and after changes.

📚 Module 5 Complete!

#	Article	Core Concept
5.1	Performance Toolkit	Profiling tools, the seven optimization strategies, th
5.2	Code Splitting Deep Dive	Dynamic import, bundle analysis, tree shaking, Susp
5.3	Virtualization	react-window, infinite scroll, when to virtualize
5.4	Memoization Strategies	When memoization helps, when it hurts, the decisic
5.5	Web Vitals & Bundle Optimization	LCP/INP/CLS, CSR vs SSR vs SSG, image optimizatio

QUICK CHECK — no marks, just for you.

1. Which Core Web Vital replaced First Input Delay (FID) in March 2024, and what does it measure?

- ◯ CLS (Cumulative Layout Shift) — how much the layout shifts unexpectedly
- ◯ INP (Interaction to Next Paint) — how fast the page responds to user input
- ◯ TTFB (Time to First Byte) — how fast the server sends the first byte
- ◯ LCP (Largest Contentful Paint) — how fast the main content becomes visible

Check answer

2. A teammate writes `import _ from 'lodash'; _.debounce(fn, 300);` and says the bundler will tree shake away the rest of lodash. Why is this reasoning wrong according to the lesson?

Tree shaking only runs in development builds, so production keeps the whole library either way

- Tree shaking works on any import style, but only after the bundle analyzer has been run to mark unused exports
- debounce is a side-effectful function, so bundlers always exclude it from tree shaking regardless of import style
- A default import of the whole library pulls in the entire package; tree shaking needs named imports from an ES module package like lodash-es, or a direct module import such as lodash/debounce

Check answer

3. With SSR, the HTML for a page arrives and is visible almost immediately, yet clicking a button does nothing for a moment. What explains this gap?

- The browser blocks input events until every image on the page has finished decoding
- The page only becomes fully interactive once hydration completes, because that is when React attaches event listeners and state to the server-rendered HTML
- The server must finish a second render pass before it will accept any client events
- SSR pages disable all event handlers until the first client-side route change occurs

Check answer

‹ Previous
Memoization Strategies & When NOT to Optimize

Next ›
Container/Presentational Pattern

React.js Interview Kit

LEARN MORE

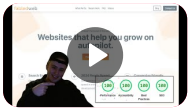
ARTICLES & DOCS

- **Web Vitals**
  web.dev
- **Tree shaking (MDN glossary)**
  MDN

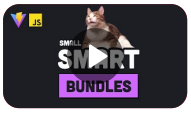
VIDEOS



- **Tree shaking — optimize bundle size**
  YouTube



- **Perfect web vitals (Next.js)**
  YouTube



- **Make your JS bundle smaller**
  YouTube

Hand-picked references for this lesson.